

Lecture 13

Kendra Burbank

Multivariate Newton's method

In the following we will assume that $f : \Omega \rightarrow \mathbb{R}$ is a strictly convex function although the method also works (with slight modification of the stopping condition) as long as f has invertible Hessian.

1. Compute the Newton step $\mathbf{p}_k$ by solving the linear system.
2. Compute the Newton decrement $\lambda_k^2 = -\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$.
3. Check stopping condition: $\lambda_k^2/2 \leq \varepsilon$. If met, stop.
4. (Optional) Perform backtracking line search starting from $\alpha = 1$ to find an acceptable α_k .
5. Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$. Repeat.

Algorithm 5 Multivariate Newton's Method

Require: $\mathbf{x}_0 \in \Omega, \varepsilon > 0$ (small)

procedure NEWTON($\mathbf{x}_0, \varepsilon$)

$k \leftarrow 0$

 repeat

 Newton step: $\mathbf{p}_k \leftarrow -[\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$

 Newton decrement: $\lambda_k^2 \leftarrow \nabla f(\mathbf{x}_k)^\top [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$

 Stopping condition: quit if $\lambda_k^2/2 \leq \varepsilon$.

 (Optional) line search: call backtracking line search to

 find α_k , or just use $\alpha_k = 1$

 Update: $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{p}_k$; $k \leftarrow k + 1$

 return $\mathbf{x}_* := \mathbf{x}_k$

end procedure

The stopping condition

The stopping condition in terms of λ_k^2 seems a bit strange but it is actually entirely consistent with the stopping condition in steepest descent.

Let's see how! But first, we need to define a few terms.

Definition (Mahalanobis norm)

For any $A \in \mathbb{S}_{++}^n$, the Mahalanobis norm of $\mathbf{x} \in \mathbb{R}^n$ is

$$\|\mathbf{x}\|_A^2 = \mathbf{x}^\top A \mathbf{x}$$

The norm

$$\|\mathbf{x}\|_{A^{-1}}^2 = \mathbf{x}^\top A^{-1} \mathbf{x}$$

is called the energy norm.

If $A = I_n$, this is the standard 2-norm on $\mathbb{R}^n$.

If A is a diagonal matrix

$$W = \begin{bmatrix} w_1 & & \\ & \ddots & \\ & & w_n \end{bmatrix}, \quad w_1, \dots, w_n > 0$$

then $\|\mathbf{x}\|_W = \sqrt{w_1 x_1^2 + \dots + w_n x_n^2}$ is the weighted 2-norm.

Note that in steepest descent, we used the 2-norm of the gradient

$$\|\nabla f(\mathbf{x}_k)\|$$

to decide when to stop.

Here, with Newton method, we used the *Mahalanobis norm* of the gradient with $A = \nabla^2 f(\mathbf{x}_k)^{-1}$, i.e.,

$$\|\nabla f(\mathbf{x}_k)\|_{\nabla^2 f(\mathbf{x}_k)^{-1}} = \nabla f(\mathbf{x}_k)^\top [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$$

to decide when to stop.

In other words, for Newton, we use a different norm at every different point $\mathbf{x}_k$ and the norm is dependent on the choice of f .

Newton's method for systems of nonlinear equations

Until now, Newton's method was used to find roots of a single function $f(\mathbf{x}) = 0$.

Now we will use it to find roots of a system of n nonlinear equations $F(\mathbf{x}) = \mathbf{0}$, where $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$.

This section extends Newton's method, previously discussed for optimization (finding minima/maxima by solving $\nabla f(\mathbf{x}) = \mathbf{0}$), to solve general systems of nonlinear equations $F(\mathbf{x}) = \mathbf{0}$, where $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$. It introduces the Jacobian matrix $JF(\mathbf{x})$ as the multi-dimensional analog of the derivative and shows how it's used in the Newton iteration. The method's application is also demonstrated for matrix-valued functions, specifically for finding the matrix square root.

Solving systems of equations is a fundamental problem in many scientific and engineering disciplines. When these equations are nonlinear, direct analytical solutions are rare, necessitating numerical methods. Newton's method provides a powerful and often rapidly converging approach by locally approximating the nonlinear system with a linear one at each iteration. The extension to matrix functions like $X^2 - A = 0$ highlights the method's versatility beyond vector-valued problems.

Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$; Newton's method can be used to numerically find $\mathbf{x}_* \in \mathbb{R}^n$ such that $F(\mathbf{x}_*) = \mathbf{0}$ by setting

$$\mathbf{p}_k \leftarrow -[JF(\mathbf{x}_k)]^{-1} F(\mathbf{x}_k),$$

So the Newton iteration is given by

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\mathbf{J}F(\mathbf{x}_k)]^{-1} F(\mathbf{x}_k).$$

Newton's method for optimization is just a special case: we are trying to find the minima of a function f , which means we are looking for the roots of the gradient $\nabla f(\mathbf{x}) = \mathbf{0}$.

So we think of $F(\mathbf{x}) = \nabla f(\mathbf{x})$ where f is the objective function. Thus $\mathbf{J}F = \mathbf{J}(\nabla f) = \nabla^2 f$.

Example

Define $F : \mathbb{R}^2 \rightarrow \mathbb{R}^2$. by

$$F(x, y) = \begin{bmatrix} x + y - 3 \\ x^2 + y^2 - 9 \end{bmatrix}$$

Then

$$\mathbf{J}F : \mathbb{R}^2 \rightarrow \mathbb{R}^{2 \times 2}, \quad \mathbf{J}F(x, y) = \begin{bmatrix} 1 & 1 \\ 2x & 2y \end{bmatrix}$$

Suppose we start at $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \end{bmatrix}$. Then $\mathbf{p}_0$ solves

$$\mathbf{J}F(x_0, y_0) \mathbf{p}_0 = -F(x_0, y_0)$$

Solving the system of equations,

$$\mathbf{p}_0 = \begin{bmatrix} -13/8 \\ -11/8 \end{bmatrix}.$$

The iteration step is

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + \mathbf{p}_0 = \begin{bmatrix} -0.625 \\ 3.625 \end{bmatrix}$$

Now $\mathbf{p}_1$ solves

$$JF(x_1, y_1) \mathbf{p}_1 = -F(x_1, y_1), \quad \Rightarrow \quad \begin{bmatrix} 1 & 1 \\ -5/4 & 29/4 \end{bmatrix} \mathbf{p}_1 = \begin{bmatrix} 0 \\ 145/32 \end{bmatrix}.$$

Hence

$$\mathbf{p}_1 = \begin{bmatrix} 145/272 \\ -145/272 \end{bmatrix},$$
$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + \mathbf{p}_1 \approx \begin{bmatrix} 0.0092 \\ 3.092 \end{bmatrix} \approx \begin{bmatrix} 0 \\ 3 \end{bmatrix}.$$

In the event when we have a matrix-variate function $F : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \times n}$, there is no longer a Jacobian matrix.

The derivative $DF(X) : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \times n}$ is a linear map

But, as long as it is an invertible linear map (i.e., bijective, or equivalently, $\text{rank } DF(X) = mn$ as a linear map), we can still do Newton's method.

Example (Matrix Square Root)

For any $A \in \mathbb{S}_{++}^n$, there exists a unique $X \in \mathbb{S}_{++}^n$ such that $X^2 = A$. This is called the matrix square root of A and often denoted $A^{1/2}$ or $\sqrt{A}$. Its existence is easy to see from the eigenvalue decomposition: If $A = Q\Lambda Q^\top$ is the eigendecomposition of A , then $X = Q\sqrt{\Lambda}Q^\top$ where $\sqrt{\Lambda} := \text{diag}(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_n})$. Computing eigenvalue decomposition is expensive, can we find $A^{1/2}$ without first finding eigenvalue decomposition of A ?

We will use the multivariate Newton's method to find $A^{1/2}$. We numerically solve $f(X) = X^2 - A = 0$. Recall that F is differentiable everywhere, and $DF(X) : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}^{n \times n}$ is given by

$$[DF(X)](H) = XH + HX$$

Hence

$$\begin{aligned} [DF(X_k)](P_k) &= X_k P_k + P_k X_k, \\ -F(X_k) &= A - X_k^2. \end{aligned}$$

Since $[DF(X_k)](P_k) = -F(X_k)$, the Newton iterations are given by

$$\begin{cases} X_k P_k + P_k X_k = A - X_k^2, \\ X_{k+1} = X_k + P_k \end{cases}$$

We would like to simplify this to get P_k explicitly.

Turn out that all we need is to choose our initial guess X_0 to commute with A , because of the following: if our initial guess commutes with A , and all subsequent iterates X_k remain in $\mathbb{S}_{++}^n$, then all X_k will commute with A .

Lemma

Suppose all $X_k \in \mathbb{S}_{++}^n$. If $X_0 A = A X_0$, then $X_k A = A X_k$ for all $k = 0, 1, \dots$. Proof. Since $A X_0 = X_0 A$ and $X_0 \in \mathbb{S}_{++}^n$ is invertible, we must also have $X_0^{-1} A = A X_0^{-1}$. If we assume $A P_0 = P_0 A$, then we get $P_0 := \frac{1}{2} (X_0^{-1} A - X_0)$ and indeed it is easy to verify that this expression satisfies

$$X_0 P_0 + P_0 X_0 = \frac{1}{2} (A - X_0^2 + X_0^{-1} A X_0 - X_0^2) = A - X_0^2$$

Our induction hypothesis is therefore

$$\begin{cases} AX_k = X_k A \\ P_k = \frac{1}{2} (X_k^{-1} A - X_k), \end{cases}$$

which we showed is true for $k = 0$ and, if true for any k , must hold for $X_{k+1} = X_k + P_k$:

$$\begin{aligned}
AX_{k+1} &= AX_k + AP_k = AX_k + \frac{1}{2} (AX_k^{-1}A - AX_k) \\
&= X_k A + \frac{1}{2} (X_k^{-1}A^2 - X_k A) \\
&= X_k A + \frac{1}{2} (X_k^{-1}A - X_k) A = X_k A + P_k A = X_{k+1} A
\end{aligned}$$

and $P_{k+1} = \frac{1}{2} (X_{k+1}^{-1}A - X_{k+1})$ is well-defined since we assume that $X_{k+1} \in \mathbb{S}_{++}^n$.

Note that this induction proof is a bit more subtle than your routine ones - it requires you to choose an inductive statement that contains more than what you set out to prove. Specifically, the inductive statement contains P_k and X_k , not just X_k .

It follows from the proof that Newton's method for the matrix square root can be written as

$$\begin{cases} X_0 = A \text{ or } I \\ X_{k+1} = \frac{1}{2} (X_k + X_k^{-1} A) \end{cases}$$

For the stopping condition, we may just use $\|X_k^2 - A\| < \varepsilon$ for some small ε . Recall Newton's method for square roots of positive real numbers: to solve $x^2 = a$, we apply

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right)$$

It is easy to see that the matrix iteration is a generalization of this.

A question: does this method always converge?

That is, for any X_0 , do we always $\lim_{k \rightarrow \infty} X_k = \sqrt{A}$?

Not always. Just like in the univariate case, $\lim_{k \rightarrow \infty} x_k = \sqrt{a}$ only if x_0 is sufficiently close to the solution. Hence we need X_0 sufficiently close to the solution in order to ensure convergence.

Gauss-Newton method for nonlinear least squares

In a linear least squares problem, aka linear regression, we optimize

$$\min_{\mathbf{x} \in \mathbb{R}^n} \sum_{i=1}^m (\mathbf{a}_i^\top \mathbf{x} - b_i)^2 = \min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|^2$$

where $A \in \mathbb{R}^{m \times n}$ has rows $\mathbf{a}_1^\top, \dots, \mathbf{a}_m^\top$ and $\mathbf{b} \in \mathbb{R}^m$ has entries $b_1, \dots, b_m$.

You will see how to reduce this to a linear system

$$A^\top A \mathbf{x} = A^\top \mathbf{b}$$

in the last homework.

A nonlinear least squares problem is one where we replace the residual $F(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$ by a nonlinear function:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \sum_{i=1}^m f_i(\mathbf{x})^2 = \min_{\mathbf{x} \in \mathbb{R}^n} \|F(\mathbf{x})\|^2$$

where $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is

$$F(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix}$$

Note that

$$F(\mathbf{x}) = \mathbf{0} \quad \Leftrightarrow \quad \min_{\mathbf{x} \in \mathbb{R}^n} \|F(\mathbf{x})\|^2 = 0$$

so solving a system of nonlinear equations is also an optimization problem, in fact, a nonlinear least squares problem.

This is hardly surprising since every problem in the world is an optimization problem!

Suppose we want to minimize $f(\mathbf{x}) = \|F(\mathbf{x})\|^2 = \sum_{i=1}^m f_i(\mathbf{x})^2$, where $F(\mathbf{x})$ is a vector of m nonlinear functions $f_i(\mathbf{x})$.

One way is to use Newton's method directly on $f(\mathbf{x})$. This requires computing the gradient $\nabla f(\mathbf{x})$ and the Hessian $\nabla^2 f(\mathbf{x})$.

The gradient is $\nabla f(\mathbf{x}) = 2JF(\mathbf{x})^\top F(\mathbf{x})$.

What about the Hessian? It's pretty complex:

Gradient: $\nabla f(\mathbf{x}) = 2JF(\mathbf{x})^\top F(\mathbf{x})$

First, let's write out what $JF(\mathbf{x})^\top F(\mathbf{x})$ means in terms of components:

$JF(\mathbf{x})$ is the Jacobian matrix where each row i is $\nabla f_i(\mathbf{x})^\top$

So $JF(\mathbf{x})^\top$ has columns that are $\nabla f_i(\mathbf{x})$

$F(\mathbf{x})$ is the vector of $f_i(\mathbf{x})$ values

The product $JF(\mathbf{x})^\top F(\mathbf{x})$ can be written as:

$\sum_{i=1}^m f_i(\mathbf{x}) \nabla f_i(\mathbf{x})$ Now, to get the Hessian, we need to take the derivative of this sum.

Using the product rule:

$$\frac{\partial}{\partial x_j} \sum_{i=1}^m f_i(\mathbf{x}) \nabla f_i(\mathbf{x}) = \sum_{i=1}^m \left(\frac{\partial f_i(\mathbf{x})}{\partial x_j} \nabla f_i(\mathbf{x}) + f_i(\mathbf{x}) \frac{\partial}{\partial x_j} \nabla f_i(\mathbf{x}) \right)$$

$$\frac{\partial}{\partial x_j} \sum_{i=1}^m f_i(\mathbf{x}) \nabla f_i(\mathbf{x}) = \sum_{i=1}^m \left(\frac{\partial f_i(\mathbf{x})}{\partial x_j} \nabla f_i(\mathbf{x}) + f_i(\mathbf{x}) \frac{\partial}{\partial x_j} \nabla f_i(\mathbf{x}) \right)$$

This gives us two terms: First term: $\sum_{i=1}^m \frac{\partial f_i(\mathbf{x})}{\partial x_j} \nabla f_i(\mathbf{x})$

Second term: $\sum_{i=1}^m f_i(\mathbf{x}) \frac{\partial}{\partial x_j} \nabla f_i(\mathbf{x})$

The first term can be rewritten as $JF(\mathbf{x})^\top JF(\mathbf{x})$ because: $\frac{\partial f_i(\mathbf{x})}{\partial x_j}$ is the (i, j) entry of $JF(\mathbf{x})$ $\nabla f_i(\mathbf{x})$ is the i th column of $JF(\mathbf{x})^\top$

The second term is $\sum_{i=1}^m f_i(\mathbf{x}) \nabla^2 f_i(\mathbf{x})$ because: $\frac{\partial}{\partial x_j} \nabla f_i(\mathbf{x})$ is the j th column of the Hessian of f_i

Putting it all together and multiplying by 2 (from the original gradient expression):

$$\nabla^2 f(\mathbf{x}) = 2JF(\mathbf{x})^\top JF(\mathbf{x}) + 2\sum_{i=1}^m f_i(\mathbf{x})\nabla^2 f_i(\mathbf{x})$$

The Gauss-Newton method makes an approximation: it *drops* the second term in the Hessian, $2 \sum_{k=1}^m f_k(\mathbf{x}) \nabla^2 f_k(\mathbf{x})$. So, the approximate Hessian is $H_{GN} = 2JF(\mathbf{x})^\top JF(\mathbf{x})$.

Why is this a reasonable approximation? 1. If the functions $f_k(\mathbf{x})$ are nearly linear, their second derivatives $\nabla^2 f_k(\mathbf{x})$ will be small. 2. If the model fits the data well, the residuals $f_k(\mathbf{x})$ will be small near the solution. In either case, the dropped term might be small compared to $2JF(\mathbf{x})^\top JF(\mathbf{x})$.

Gauss-Newton method is a special algorithm designed specifically for objective function of the form $f(\mathbf{x}) = \|F(\mathbf{x})\|^2$, i.e., for nonlinear least squares problems

Unlike just applying Newton method directly, Gauss-Newton avoids second derivatives.

If $m = n$, then the Gauss-Newton iteration is simply

$$\mathbf{x}_{k+1} = \mathbf{x}_k - JF(\mathbf{x}_k)^{-1} F(\mathbf{x}_k),$$

assuming that the Jacobian $JF(\mathbf{x}_k) \in \mathbb{R}^{n \times n}$ is invertible. Unsurprisingly it reduces to the same iteration in (14.1) for solving $F(\mathbf{x}) = \mathbf{0}$.

More generally for any $m \geq n$, the Gauss-Newton iteration is given by

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \left[JF(\mathbf{x}_k)^\top JF(\mathbf{x}_k) \right]^{-1} JF(\mathbf{x}_k)^\top F(\mathbf{x}_k),$$

assuming that the Jacobian $JF(\mathbf{x}_k) \in \mathbb{R}^{m \times n}$ has rank n .

Note that $A \in \mathbb{R}^{m \times n}$ has rank n iff $A^\top A \in \mathbb{R}^{n \times n}$ is invertible.

The matrix $(A^\top A)^{-1} A^\top \in \mathbb{R}^{n \times m}$ is called the Moore-Penrose pseudoinverse of A , a sort of “inverse” for a rectangular matrix.

This is very much related to (14.6). In particular, if $m = n$ and A is invertible, then $(A^\top A)^{-1} A^\top = A^{-1}$, so (14.8) reduces to (14.7).

Gauss-Newton method is an approximation of Newton method and can be derived as follows.

Note that by chain rule,

$$\nabla f(\mathbf{x}) = 2 \mathbf{J}F(\mathbf{x})^\top F(\mathbf{x}) \quad \text{and} \quad \nabla^2 f(\mathbf{x}) \approx 2 \mathbf{J}F(\mathbf{x})^\top \mathbf{J}F(\mathbf{x})$$

where the approximation is obtained by dropping the term in red:

$$\frac{\partial^2 f}{\partial x_i \partial x_j} = 2 \sum_{k=1}^m \left(\frac{\partial f_k}{\partial x_i} \frac{\partial f_k}{\partial x_j} + \textcolor{red}{f_k} \frac{\partial^2 f_k}{\partial x_i \partial x_j} \right)$$

which is valid if the term in red is small (e.g., when f_k is nearly linear).

Now observe that by (14.9), the Newton direction

$$\nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k) \approx \left[JF(\mathbf{x}_k)^\top JF(\mathbf{x}_k) \right]^{-1} JF(\mathbf{x}_k)^\top F(\mathbf{x}_k)$$

which gives us (14.8).

Stochastic gradient descent

More generally many optimization problems in machine learning take a form like

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) = \min_{\mathbf{x} \in \mathbb{R}^n} \sum_{i=1}^m f_i(\mathbf{x})$$

where m is the size of the sample or training set or observations (huge in modern applications) and n is the number of parameters (also huge in modern applications) to be estimated from the observations.

Take a simple binary classification problem into two classes represented by -1 and +1 respectively. Examples include the following regularized least squares problem:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{m} \sum_{i=1}^m (y_i - \mathbf{a}_i^\top \mathbf{x})^2 + \frac{\lambda}{2} \|\mathbf{x}\|^2;$$

L -regularized binary logistic regression:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{m} \sum_{i=1}^m \log(1 + e^{-y_i(\mathbf{a}_i^\top \mathbf{x})}) + \frac{\lambda}{2} \|\mathbf{x}\|^2;$$

L -regularized squared hinge loss:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{m} \sum_{i=1}^m \left[(1 - y_i \mathbf{a}_i^\top \mathbf{x})_+ \right]^2 + \frac{\lambda}{2} \|\mathbf{x}\|_2^2$$

nonconvex support vector machine with a sigmoid loss:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{m} \sum_{i=1}^m (1 - \tanh(y_i \mathbf{a}_i^\top \mathbf{x})) + \frac{\lambda}{2} \|\mathbf{x}\|_2^2$$

Here $\lambda > 0$ is a regularization parameter, $y_1, \dots, y_m \in \{-1, +1\}$ are training labels, and $\mathbf{a}_1, \dots, \mathbf{a}_m \in \mathbb{R}^n$ are observations whose values are known.

Stochastic gradient descent is the simple procedure where the usual updates

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k) = \mathbf{x}_k - \eta_k \left[\frac{1}{m} \sum_{i=1}^m \nabla f_i(\mathbf{x}_k) \right], \quad \alpha_k = \eta_k/m$$

is replaced by

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \left[\frac{1}{|S_k|} \sum_{i \in S_k} \nabla f_i(\mathbf{x}_k) \right]$$

where $S_k \subseteq \{1, \dots, m\}$ is a small randomly chosen subset called a minibatch.

In this context, the stepsize η_k is often called the learning rate.

In usual steepest descent, we want

$$\lim_{k \rightarrow \infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$$

In stochastic gradient descent, noting that $\nabla f_i(\mathbf{x}_k)$ is now a random variable because i is, we want

$$\lim_{k \rightarrow \infty} \mathbb{E}[\nabla f_i(\mathbf{x}_k)] = \mathbf{0}$$